

Install the MongoDB MCP Server in VS Code

For this install I am using an older Ubuntu 24.04 and Node version 24.14.0

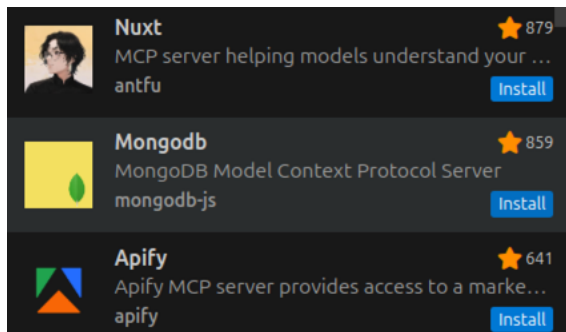
Before you begin this process, you must have MongoDB installed on your Linux system. You must have `npm` installed in your Linux system before proceeding.

If you are working with this document after installing GitHub MCP Server, some of these steps will be redundant. You will have to skip a few of them. You will be editing the same `mcp.json` file, do not create a new one.

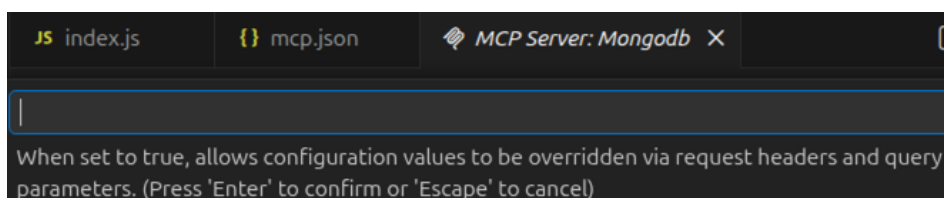
All three installations are being done in **June of 2026** and may have changed by the time the bootcamps are aired.

Part 1 – Extension Installation

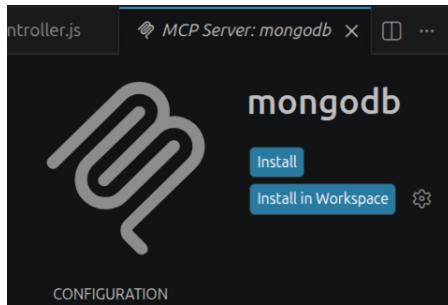
1. Open the Extensions panel (left of main editor window). You will see a search bar at the top of that window, begin typing `@mcp`. This will trigger the MCP list to show up.
2. Choose `Mongodb` and then click the blue `Install` button. You may be asked to sign in, if you have not already done so.



3. You will see the quick pick tool appear, just hit `Esc`



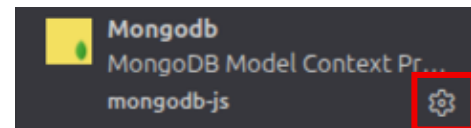
4. You may have to hit *Install* button again:



5. If you look at the `mcp.json` file, it should list the MongoDB MCP Server there along with GitHub and Postman

```
},  
  "gallery": "https://api.mcp.github.com",  
  "version": "2.6.0"  
},  
✓Running | Stop | Restart | 26 tools | More...  
"io.github.mongodb-js/mongodb-mcp-server": {  
  "type": "stdio",  
  "command": "npx",  
  "args": [  
    "mongodb-mcp-server@1.5.0",  
    "--allowRequestOverrides",  
    "${input:allowRequestOverrides = }",  
    "--apiClientId"  
  ]  
}
```

You can see any installed MCP Server by clicking on the respective icon:



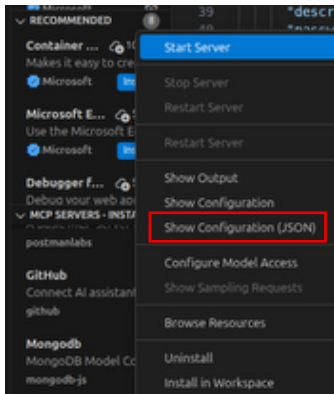
Note: your version may be different. You may even get an error when you try to start the server. Either of these situations means that you have to work through them using Chat or the internet generally.

6. We do not need all the lines you may be seeing on your configuration. Let's remove most of it and replace it with the lines you see below.

```
        "gallery": "https://api.mcp.github.com",
        "version": "2.6.0"
    },
    "io.github.mongodb-js/mongodb-mcp-server": {
        "type": "stdio",
        "command": "npx",
        "args": [
            "-y",
            "mongodb-mcp-server",
            "--connectionString",
            "mongodb://localhost:27017"
        ],
        "env": {
            "MDB_MCP_READ_ONLY": "true"
        },
        "gallery": "https://api.mcp.github.com",
        "version": "1.8.0"
    }
}
```

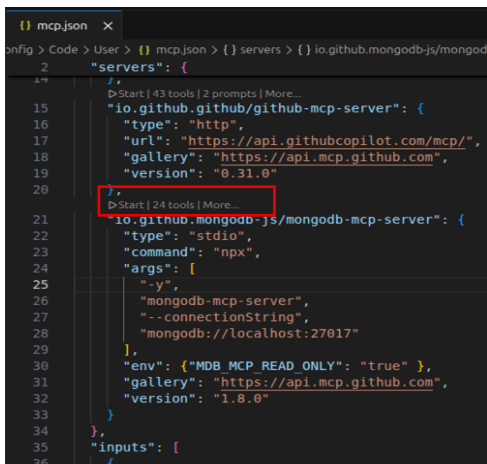
Remember, your configuration may be slightly different but the main parts here are the `args`, `env` and the first two key/value pairs. The versions may have changed. If you prefer to use a cloud-based DB like MongoDB Atlas, your config here will be different. If you are using local MongoDB, make sure the service is running in your Linux OS.

7. The configuration above will get the connection string from the `args` section. In the next section of the bootcamp, we will have VS Code look in an `.env` file in the root of our project folder for the connection string.
8. Start the MCP server and before you begin to ask questions (prompts). One option is to open the extensions bar and go to MongoDB and click on the gear icon. That will open an options list. One of those options is to Start Server:

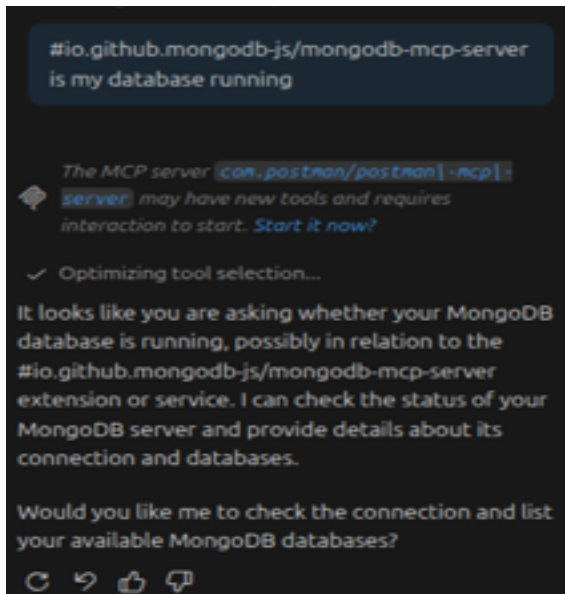


Remember you can also start the server directly from the mcp.json file.

9. Another option to start is directly in the mcp.json file. Each MCP Server you install will have this option at the start of the configuration piece:



10. You can also start the MCP server by asking simple questions. I asked the server if



my database is running and the response is shown in this image.

Based on the response from Copilot I responded with a “yes”.

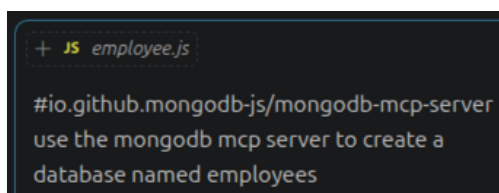
If you now look at the `mcp.json` file in the VS Code editor, you will see that the MCP Server is “Running”.

You can now close that `mcp.json` file. You won’t need it again unless you install another MCP Server. We will install the Postman MCP Server in the next section of the bootcamp.

Part 2 – Using the MongoDB MCP Server

Lets now create a database, then a collection and add a few documents. For this part make sure your local MongoDB service is running and you have entered via the MongoD shell.

1. In the chat window:



In my case, I had to reconfigure the MongoDB MCP Server to have write access.

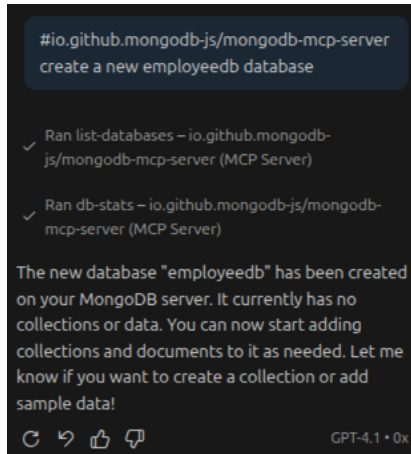
Note, you must name the MongoDB database the same as you have it in the `employees.js`

file. Look for the `mongoose.connect()` method and the connection string.

You may have to give permissions, again this may change based on when you are participating in this bootcamp.

If you are not familiar with No-SQL databases, a *collection* is the equivalent of a table and a *document* is a record. At this point, we may have differing results based on our configuration. Right now we are trying to get the server to start and available for write access from the MCP Server.

2. The response:



In this image the database is called `employeeDB` but I recently changed it to `employees` based on MongoDB's naming convention.

Below is my terminal window showing before and after the prompt:

```
test> show dbs
admin    40.00 KiB
config  48.00 KiB
local   76.00 KiB
test> show dbs
admin    40.00 KiB
config  72.00 KiB
employees 8.00 KiB
local   76.00 KiB
```

3. In some cases your MCP Server may not have “write” access. If this is the case you must disable the key value pair in the env section:

```
"env": {
  "MDB_MCP_READ_ONLY": "false"
},
```

If you made this change you MUST restart the server.

4. Now just continue:

In the `employeeDB` database you just created, add a collection called `ft_employees`

Notice that I did not have to use the hashtag. The system is smart enough to know that by now I am talking about MongoDB.

5. Try adding a document (record):

Add this document to the `ft_employees` collection: `{empName : "Joe", empPass : "1234" }`

6. Try adding a new document based on the last one:

Add a new document with `empName` of `Mary`, use the same value for `password`

Notice you do not have to go into details like #5, the LLM will take its cues from the previous prompt,

7. Now you can verify via the MongoDB shell that everything was done based on your instructions:

```
test> show dbs
admin      40.00 KiB
config     48.00 KiB
employees  40.00 KiB
local      72.00 KiB
test> use employees
switched to db employees
employees> db.ftEmployees.find({})
[
  {
    _id: ObjectId('69caa2b7ccee9bce79f3b9ec'),
    empName: 'Joe',
    empPass: '1234'
  },
  {
    _id: ObjectId('69caa340ccee9bce79f3b9ed'),
    empName: 'Mary',
    empPass: 'Mary'
  }
]
employees>
```

Part 3 – Running Tasks using the MCP Server

Lets now execute more complex commands on the MongoDB database using just our Chat window provided by Copilot.

1. Now just continue:

Show me the MongoDB command before you execute it. Add a new document based on the previous two for employee Bob but in this case add a time stamp

Unfortunately, the tool did pause but then executed the command. I was not able to verify the command but this is the result of adding Bob's document:

```
  _id: ObjectId('69caa340ccee9bce79f3b9ed'),
  empName: 'Mary',
  empPass: 'Mary'
},
{
  _id: ObjectId('69caaadccee9bce79f3b9ee'),
  empName: 'Bob',
  empPass: 'Bob',
  timestamp: ISODate('2026-03-30T16:54:51.000Z')
}
```

2. Try changing the existing documents to match Bob's:

Using the mongodb mcp server, add timestamps to any document that does not have one

This time I was able to capture the command:

```
db.ftEmployees.updateMany(
  { timestamp: { $exists: false } },
  { $currentDate: { timestamp: true } }
)
```

```
employees> db.ftEmployees.find({})
[
  {
    _id: ObjectId('69caa2b7ccee9bce79f3b9ec'),
    empName: 'Joe',
    empPass: '1234',
    timestamp: ISODate('2026-03-30T17:00:29.808Z')
  },
  {
    _id: ObjectId('69caa340ccee9bce79f3b9ed'),
    empName: 'Mary',
    empPass: 'Mary',
    timestamp: ISODate('2026-03-30T17:00:29.808Z')
  },
  {
    _id: ObjectId('69caaadccee9bce79f3b9ee'),
    empName: 'Bob',
    empPass: 'Bob',
    timestamp: ISODate('2026-03-30T16:54:51.000Z')
  }
]
```

3. Lets update Bob's password from 1234 to just "bob":

Using the mongodb mcp server, change the password for Bob from 1234 to just "bob". show me the update command.

This time I was able to capture the update statement and copy the code itself:

```
db.ftEmployees.updateMany(
  { empName: "Bob", empPass:
  { $set: { empPass: "bob" } }
)
```

```
db.ftEmployees.updateMany(
  { empName: "Bob", empPass: "Bob" },
  { $set: { empPass: "bob" } }
)
```

4. Now we can add a "weights" collection and associate weights to employees:

Using the mongodb mcp server, create a new collection called "employee_weights". In that new collection create a weight document based on Bob's _id. The new weight document should contain Bobs unique id, a made up weight between 20 and 200 and a date that the weight was uploaded. It can be any date in the last 3 months.

This is the result of this Prompt:

```
{
  _id: ObjectId('69caadf3cee9bce79f3b9ee'),
  empName: 'Bob',
  empPass: 'bob',
  timestamp: ISODate('2026-03-30T16:54:51.000Z')
}
]
employees> db.employeeWeights.find({})
[
  {
    _id: ObjectId('69cab7fec3ee9bce79f3b9ef'),
    employeeId: ObjectId('69caadf3cee9bce79f3b9ee'),
    weight: 178,
    uploadedAt: ISODate('2026-02-14T10:30:00.000Z')
  }
]
employees> █
```

Notice that in the employee_weights collection, the employeeId is the same as that in Bob's document in the ft_employees collection.

Note: We will create a .env file in the Postman section. There we will add the connection string for this MongoDB project into the .env file. We will then adjust the mcp.json file to pick up the value from there.

Optional section

5. Now you can ask all kinds of questions about the database:
 - a. How large is the employees database
 - b. Show date and time of the last entry into the ftEmployees collection
 - c. Analyze the two documents and come up with a JS enum for them
 - d. Convert all documents into a .csv file and put the file into a folder called data in the root of my app

```
employees> db.allEmployees.find()
[
  {
    _id: ObjectId('6a205db963457094b68ce5b0'),
    empName: 'Joe',
    empPass: '1234'
  },
  {
    _id: ObjectId('6a205db963457094b68ce5b1'),
    empName: 'Mary',
    empPass: '1234'
  },
  {
    _id: ObjectId('6a205db963457094b68ce5b2'),
    empName: 'John',
    empPass: '1234'
  },
  {
    _id: ObjectId('6a205db963457094b68ce5b3'),
    empName: 'Alice',
    empPass: '1234'
  },
  {
    _id: ObjectId('6a205db963457094b68ce5b4'),
    empName: 'Bob',
    empPass: '1234'
  }
]
employees> █
```

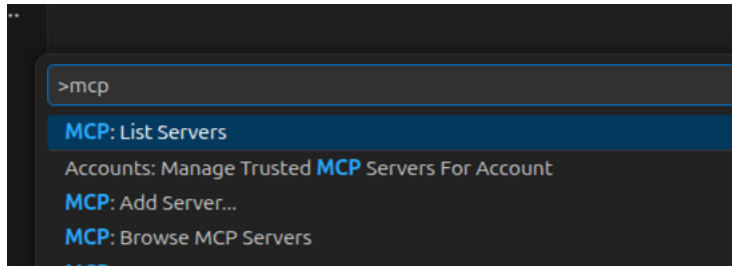
- e. Create a new collection called regEmployees and add three dummy documents based on the documents in ftEmployees
- f. Now merge both sets of documents into a new collection called allEmployees

Appendix A – VS Code and the MCP Server List

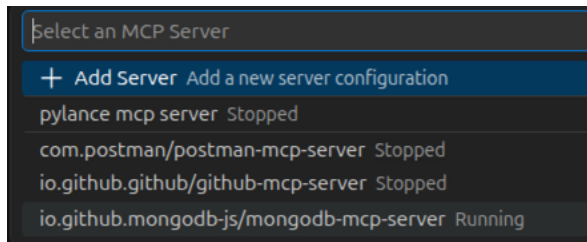
To open the `mcp.json` file in VS Code, the fastest and most reliable method is using the Command Palette. This ensures you are opening the specific file that GitHub Copilot is actually reading.

1. The Command Palette Method (Recommended)

1. Press Ctrl + Shift + P (Windows/Linux) or Cmd + Shift + P (Mac).
2. Type MCP: Open User Configuration and press Enter.
 - This will immediately open your global `mcp.json` file in the editor.

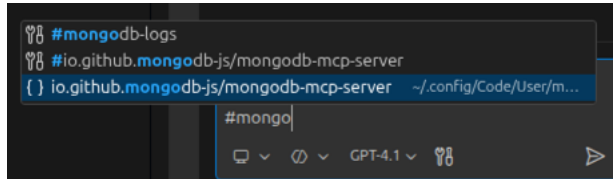


After choosing MCP:List Servers, this is what I see:



Appendix B – The Hashtag Options

The three options you see in the dropdown represent different ways of interacting with the MongoDB MCP (Model Context Protocol) server. In VS Code Copilot, the icons and prefixes tell you whether you are referencing a tool, a resource, or the server configuration itself.



Here is the breakdown of what each option does:

1. The "Wrench" icon (#io.github.mongodb...)

Type *MCP Tool* if you want to invoke a specific action. It tells Copilot to use the executable functions provided by the server (e.g., list-databases, query-collection, or insert-document).

Use this when you have a specific task in mind, like "List the collections in my 'production' database using #io.github.mongodb...."

2. The "Braces" icon ({} io.github.mongodb...)

Type *MCP Resource / Config* to provide raw data or schema context. This attaches the "static" information from the server—such as a database schema definition or a specific log file—as background context for the AI.

Use this when you want Copilot to "read" your database structure before you ask a general question, like "Looking at {} io.github.mongodb..., how should I structure a query for the users collection?"

3. The #mongodb-logs option

Type *Log Resource* which specifically attaches the runtime logs of the MCP server itself. Only choose this if the server is failing to connect or throwing errors and you want Copilot to help you debug the connection.

```
axle@pc0480:~$ sudo systemctl status mongod
○ mongod.service - MongoDB Database Server
   Loaded: loaded (/usr/lib/systemd/system/mongod.service; disabled; preset:
   Active: inactive (dead)
     Docs: https://docs.mongodb.org/manual

axle@pc0480:~$ sudo systemctl start mongod
axle@pc0480:~$ sudo systemctl status mongod
● mongod.service - MongoDB Database Server
   Loaded: loaded (/usr/lib/systemd/system/mongod.service; disabled; preset:
   Active: active (running) since Mon 2026-03-30 11:56:00 ADT; 6s ago
     Docs: https://docs.mongodb.org/manual
   Main PID: 17857 (mongod)
    Memory: 193.2M (peak: 276.7M)
       CPU: 534ms
    CGroup: /system.slice/mongod.service
           └─17857 /usr/bin/mongod --config /etc/mongod.conf

Mar 30 11:56:00 pc0480 systemd[1]: Started mongod.service - MongoDB Database Se
Mar 30 11:56:00 pc0480 mongod[17857]: {"t":["$date":"2026-03-30T14:56:00.600Z"]>
lines 1-12/12 (END)
```

```
sudo mongosh
```

```
test> show dbs
admin      40.00 KiB
config     48.00 KiB
local      72.00 KiB
test>
```

For the database name use camelCase but add DB at the end, employeeDB

Finally for documents use camelCase e.g. firstName